جزوه جامع

توابع در جاوا اسکریپت

از مبانی تا الگوهای پیشرفته

شامل ۲۲ فصل، مثال‌های اجرایی، تمرین و پاسخ تشریحی

تمام کدها به زبان انگلیسی و قابل اجرا در Node.js و کنسول مرورگر

آبان ۱۴۰۵

پیش‌گفتار و راهنمای استفاده

این جزوه به‌صورت مرحله‌ای و از مفاهیم پایه تا الگوهای پیشرفته، نحوه نوشتن و استفاده از توابع در زبان JavaScript را شرح می‌دهد. هر فصل شامل توضیح مفهومی، مثال کد، و نکات رایج خطا است و در پایان، تمرین‌هایی همراه با پاسخ ارائه شده است.

پیش‌نیازها

- آشنایی با متغیرها (`let`, `const`)، انواع داده، عملگرها، شرط‌ها و حلقه‌ها
- نصب Node.js یا استفاده از کنسول مرورگر (کلید F12) برای اجرای مثال‌ها

نحوه اجرای مثال‌ها

هر مثال را در یک فایل با پسوند `.js` ذخیره کنید و با دستور زیر اجرا نمایید. همچنین می‌توانید کد را مستقیماً در کنسول مرورگر وارد کنید.

```
node example.js
```

قراردادهای جزوه

- نتیجه اجرای هر خط، به‌صورت کامنت در همان خط یا خط بعد آمده است (مثلاً `// 25`).
- تمام شناسه‌ها، کامنت‌ها و رشته‌های داخل کد به زبان انگلیسی نوشته شده‌اند.
- مثال‌ها مطابق استاندارد ES2015 به بعد (ES6+) هستند.

فهرست مطالب

- فصل ۱: مفهوم تابع و جایگاه آن در جاوا اسکریپت
- فصل ۲: تعریف و فراخوانی تابع
- فصل ۳: پارامترها و آرگومان‌ها
- فصل ۴: مقدار بازگشتی (return)
- فصل ۵: Function Expression و توابع ناشناس
- فصل ۶: توابع پیکانی (Arrow Functions)
- فصل ۷: Hoisting
- فصل ۸: دامنه (Scope) و Closure
- فصل ۹: توابع مرتبه بالا و Callback
- فصل ۱۰: متدهای تابعی آرایه‌ها
- فصل ۱۱: تابع بلافاصله اجراشونده (IIFE)
- فصل ۱۲: توابع بازگشتی (Recursion)
- فصل ۱۳: کلمه this و متدهای call، apply و bind

- فصل ۱۴: متدهای شیء و توابع سازنده
- فصل ۱۵: توابع غیرهمزمان: Promise و async/await
- فصل ۱۶: توابع Generator
- فصل ۱۷: الگوهای پیشرفته در توابع
- فصل ۱۸: مدیریت خطا در توابع
- فصل ۱۹: توابع خالص و اثرات جانبی
- فصل ۲۰: اصول و بهترین روش‌ها
- فصل ۲۱: تمرین‌ها
- فصل ۲۲: پاسخ تمرین‌ها
- پیوست: جدول مقایسه انواع تابع

فصل ۱: مفهوم تابع و جایگاه آن در جاوا اسکریپت

تابع (Function) بلوکی مستقل از کد است که یک وظیفه مشخص را انجام می‌دهد و می‌توان آن را به‌جای نوشتن مجدد کد، با نام صدا زد. تقریباً هر برنامه جاوا اسکریپت از ترکیب تعداد زیادی تابع کوچک و مشخص ساخته می‌شود.

چرا از تابع استفاده می‌کنیم؟

- **جلوگیری از تکرار (اصل DRY):** منطق یک‌بار نوشته می‌شود و در چند جا استفاده می‌شود.
- **خوانایی:** نام مناسب تابع، هدف کد را بیان می‌کند.
- **نگهداری آسان:** تغییر در یک نقطه، در همه جا اعمال می‌شود.
- **قابلیت آزمون:** توابع کوچک و مستقل به‌سادگی تست می‌شوند.
- **انتزاع:** استفاده‌کننده فقط باید بداند تابع چه کاری می‌کند، نه اینکه چگونه انجام می‌دهد.

نمونه ساده: قبل و بعد از تابع

```
// Without a function: repeated code
console.log("Hello, Sara!");
console.log("Hello, Ali!");

// With a function: written once, reused many times
function greet(name) {
  console.log("Hello, " + name + "!");
}
greet("Sara");
greet("Ali");
```

اجزای یک تابع

```
//       name   parameters
function greet(name, greeting) {
  // function body
  return greeting + ", " + name + "!";   // return value
}

// Call: "Sara" and "Hello" are the arguments
const message = greet("Sara", "Hello");
console.log(message); // Hello, Sara!
```

تابع به‌عنوان شهروند درجه یک

در جاوا اسکریپت، تابع یک «مقدار» است (در واقع نوعی شیء)، به همین دلیل به آن شهروند درجه یک (First-class citizen) می‌گویند. یعنی می‌توان تابع را:

- در یک متغیر یا ویژگی شیء ذخیره کرد؛
- به‌عنوان آرگومان به تابع دیگر داد؛
- به‌عنوان نتیجه از یک تابع بازگرداند.

```
function sayHello() {
  return "Hello";
}

const alias = sayHello;                    // stored in a variable
console.log(alias());                      // Hello
console.log(typeof sayHello);              // function
console.log(sayHello instanceof Object);  // true
```

نکته: همین ویژگی پایه‌ای برای مباحثی مانند Callback، Closure و توابع مرتبه بالا است که در فصل‌های بعد بررسی می‌شوند.

فصل ۲: تعریف و فراخوانی تابع

Function Declaration

رایج‌ترین روش تعریف تابع، «اعلان تابع» است. ساختار کلی آن به شکل زیر است:

```
function functionName(param1, param2) {
  // function body
  return result;
}
```

- کلمه کلیدی `function` شروع تعریف را مشخص می‌کند.
- نام تابع باید با حرف، `_` یا `$` شروع شود و شامل فاصله نباشد.
- پارامترها داخل پرانتز و با کاما جدا می‌شوند (می‌توانند خالی باشند).
- بدنه تابع داخل آکولاد قرار می‌گیرد.

فراخوانی تابع

تعریف تابع، آن را اجرا نمی‌کند. تابع فقط زمانی اجرا می‌شود که با نام و پرانتز فراخوانی (Call) شود.

```
function add(a, b) {
  return a + b;
}

const result = add(2, 3);
console.log(result); // 5
```

تفاوت ارجاع به تابع و فراخوانی آن

```
function getTime() {
  return "12:00";
}

console.log(getTime);   // [Function: getTime]  (a reference, not a call)
console.log(getTime()); // 12:00                (the function is executed)
```

> **هشدار:** یکی از خطاهای رایج مبتدیان، فراموش کردن پرانتز هنگام فراخوانی است. نوشتن `getTime` بدون پرانتز، تابع را اجرا نمی‌کند و فقط به خود تابع اشاره دارد.

تابع بدون return

اگر تابعی مقدار بازگشتی صریح نداشته باشد، به‌صورت پیش‌فرض مقدار `undefined` برمی‌گرداند.

```
function logOnly() {
  console.log("no return statement");
}

const value = logOnly(); // prints: no return statement
console.log(value);      // undefined
```

قواعد نام‌گذاری

- از سبک camelCase استفاده کنید: `getUserName`، `calculateTotal`.
- نام تابع باید با فعل شروع شود و کاری که انجام می‌دهد را بیان کند: `renderList`، `fetchData`.
- توابعی که مقدار منطقی (Boolean) برمی‌گردانند معمولاً با `has`، `is` یا `can` شروع می‌شوند: `hasAccess`، `isValid`.
- نام‌های کلی و مبهم مانند `doIt` یا `process1` را به کار نبرید.

فصل ۳: پارامترها و آرگومان‌ها

پارامتر (Parameter) نامی است که در تعریف تابع می‌آید و آرگومان (Argument) مقداری است که هنگام فراخوانی داده می‌شود.

```
function multiply(a, b) {   // a and b are parameters
  return a * b;
}
multiply(4, 5);             // 4 and 5 are arguments
```

تعداد آرگومان‌ها

- اگر آرگومان کمتر از پارامترها باشد، پارامترهای باقی‌مانده مقدار `undefined` می‌گیرند.
- اگر آرگومان بیشتر باشد، مقادیر اضافی نادیده گرفته می‌شوند (اما از طریق rest یا `arguments` قابل دسترسی‌اند).

```
function show(a, b) {
  console.log(a, b);
}
show(1);        // 1 undefined
show(1, 2, 3);  // 1 2
```

مقدار پیش‌فرض پارامتر (Default Parameters)

```
function power(base, exponent = 2) {
  return base ** exponent;
}
console.log(power(5));     // 25
console.log(power(2, 3));  // 8
```

مقدار پیش‌فرض در زمان فراخوانی محاسبه می‌شود و می‌تواند به پارامترهای قبلی اشاره کند:

```
function createUser(name, role = "guest", id = name + "_001") {
  return { name, role, id };
}
console.log(createUser("sara"));
// { name: 'sara', role: 'guest', id: 'sara_001' }
```

هشدار: مقدار پیش‌فرض فقط وقتی اعمال می‌شود که آرگومان `undefined` باشد؛ مقادیر `0`، `null` و رشته خالی جایگزین نمی‌شوند.

```
function show(value = "default") {
  console.log(value);
}
show(undefined); // default
show(null);      // null
show(0);         // 0
```

پارامتر باقی‌مانده (Rest Parameters)

با سه نقطه (`...`) می‌توان تعداد نامحدودی آرگومان را در قالب یک آرایه واقعی دریافت کرد. پارامتر rest باید آخرین پارامتر باشد و فقط یک‌بار می‌تواند به کار رود.

```
function sum(...numbers) {
  let total = 0;
  for (const n of numbers) {
    total += n;
  }
  return total;
}
console.log(sum(1, 2, 3, 4)); // 10
console.log(sum());           // 0

function log(level, ...messages) {
  console.log(level, messages);
}
log("INFO", "a", "b"); // INFO [ 'a', 'b' ]
```

عملگر Spread هنگام فراخوانی

```
const values = [3, 9, 4];
console.log(Math.max(...values)); // 9
```

شیء arguments

در توابع معمولی (نه پیکانی)، شیء `arguments` شامل همه آرگومان‌های ارسال‌شده است. این شیء آرایه واقعی نیست و متدهایی مانند `map` ندارد؛ امروزه استفاده از rest پیشنهاد می‌شود.

```
function legacySum() {
  let total = 0;
  for (let i = 0; i < arguments.length; i++) {
    total += arguments[i];
  }
  return total;
}
console.log(legacySum(1, 2, 3)); // 6
```

Destructuring در پارامترها (پارامتر نام‌دار)

وقتی تابع پارامترهای زیادی دارد، دریافت یک شیء و باز کردن آن در همان لحظه، خوانایی را بالا می‌برد و ترتیب آرگومان‌ها اهمیت خود را از دست می‌دهد.

```
function createAccount({ name, age = 18, role = "user" } = {}) {
  return { name, age, role };
}
console.log(createAccount({ name: "Sara", role: "admin" }));
// { name: 'Sara', age: 18, role: 'admin' }
```

ارسال مقدار: مقادیر اولیه و اشیاء

- مقادیر اولیه (عدد، رشته، boolean و ...) به‌صورت کپی ارسال می‌شوند؛ تغییر آن‌ها داخل تابع بیرون را تغییر نمی‌دهد.
- برای اشیاء و آرایه‌ها، کپی ارجاع (Reference) ارسال می‌شود؛ تغییر ویژگی‌های شیء داخل تابع در بیرون هم دیده می‌شود، ولی جایگزین کردن کل متغیر پارامتر روی بیرون اثری ندارد.

```
function rename(user) {
  user.name = "Changed";      // mutates the original object
}
function reassign(user) {
  user = { name: "Brand new" }; // only the local variable changes
}

const account = { name: "Original" };
rename(account);
console.log(account.name); // Changed

account.name = "Original";
reassign(account);
console.log(account.name); // Original
```

فصل ۴: مقدار بازگشتی (return)

دستور `return` اجرای تابع را متوقف می‌کند و مقدار مشخص‌شده را به فراخوان بازمی‌گرداند. هر کدی بعد از `return` اجرا نمی‌شود.

```
function check(n) {
  return n > 0;
  console.log("never runs"); // unreachable code
}
```

بازگشت زودهنگام (Early Return / Guard Clause)

به‌جای تو در تو کردن شرط‌ها، حالت‌های نامعتبر یا ساده را در ابتدا کنار بگذارید. نتیجه، کدی تخت‌تر و خواناتر است.

```
function divide(a, b) {
  if (typeof a !== "number" || typeof b !== "number") {
    return null;
  }
  if (b === 0) {
    return null;
  }
  return a / b;
}
console.log(divide(10, 4)); // 2.5
console.log(divide(1, 0));  // null
```

بازگرداندن چند مقدار

هر تابع فقط یک مقدار برمی‌گرداند؛ برای چند مقدار از آرایه یا شیء استفاده کنید و در محل دریافت، آن را باز (Destructure) کنید.

```
function minMax(numbers) {
  return { min: Math.min(...numbers), max: Math.max(...numbers) };
}
const { min, max } = minMax([4, 9, 1, 7]);
console.log(min, max); // 1 9
```

دام نقطه‌ویرگول خودکار (ASI)

اگر بعد از `return` بلافاصله خط جدید شروع شود، جاوا اسکریپت به‌طور خودکار نقطه‌ویرگول اضافه می‌کند و تابع `undefined` برمی‌گرداند.

```
function broken() {
  return
  { ok: true };
}
function correct() {
  return {
    ok: true,
  };
}
console.log(broken());  // undefined
console.log(correct()); // { ok: true }
```

نکته: شروع مقدار بازگشتی را همیشه در همان خطی بنویسید که `return` قرار دارد.

فصل ۵: Function Expression و توابع ناشناس

تابع را می‌توان به‌صورت یک عبارت (Expression) تعریف کرد و نتیجه را در متغیر ذخیره نمود. این تابع می‌تواند بدون نام (Anonymous) یا نام‌دار باشد.

```
// Anonymous function expression
const multiply = function (a, b) {
  return a * b;
};
console.log(multiply(3, 4)); // 12

// Named function expression
const factorial = function fact(n) {
  return n <= 1 ? 1 : n * fact(n - 1);
};
console.log(factorial(5)); // 120
// "fact" is visible only inside the function itself
```

نام‌گذاری یک Function Expression به خوانایی پیام‌های خطا (Stack trace) کمک می‌کند و امکان فراخوانی تابع از داخل خودش را فراهم می‌سازد.

مقایسه Declaration و Expression

ویژگی	Function Declaration	Function Expression
نحو	`function name() {}`	`const name = function () {}`
Hoisting	کامل؛ قبل از خط تعریف هم قابل فراخوانی است	خیر؛ تا اجرای خط تعریف در دسترس نیست
نام تابع	اجباری	اختیاری
کاربرد رایج	توابع اصلی و عمومی برنامه	ذخیره در متغیر، ارسال به‌عنوان آرگومان

توابع ناشناس به‌عنوان آرگومان

```
setTimeout(function () {
  console.log("runs after 1 second");
}, 1000);

[1, 2, 3].forEach(function (item) {
  console.log(item * 2);
});
```

فصل ۶: توابع پیکانی (Arrow Functions)

توابع پیکانی (Arrow Functions) در ES2015 معرفی شدند و نحو کوتاه‌تری دارند. علاوه بر کوتاهی، رفتار آن‌ها نسبت به `this` با توابع معمولی تفاوت اساسی دارد.

شکل‌های نوشتاری

```
// Full form
const add = (a, b) => {
  return a + b;
};

// Implicit return (single expression, no braces)
const add2 = (a, b) => a + b;

// A single parameter does not need parentheses
const square = x => x * x;

// No parameters
const getPi = () => 3.14159;

// Returning an object literal: wrap it in parentheses
const makeUser = name => ({ name: name, active: true });

console.log(add(1, 2), add2(1, 2), square(5), getPi());
console.log(makeUser("Sara")); // { name: 'Sara', active: true }
```

تفاوت‌ها با توابع معمولی

ویژگی	تابع معمولی	تابع پیکانی
`this`	بسته به نحوه فراخوانی تعیین می‌شود	از محیط بیرونی به ارث می‌رسد (Lexical)
`arguments`	دارد	ندارد (از rest استفاده کنید)
استفاده با `new`	ممکن است	خطای TypeError
متد شیء	مناسب	معمولاً نامناسب
callback کوتاه	ممکن	انتخاب رایج

مزیت اصلی: this لغوی

```
const timer = {
  seconds: 0,
  start() {
    setTimeout(() => {
      this.seconds += 1;          // "this" is the timer object
      console.log(this.seconds);  // 1
    }, 100);
  },
};
timer.start();
```

هشدار: از تابع پیکانی به‌عنوان متد شیء استفاده نکنید؛ `this` در آن به شیء اشاره نمی‌کند.

```
const box = {
  value: 10,
  broken: () => this.value,       // "this" is NOT box here
  working() { return this.value; },
};
console.log(box.working()); // 10
```

فصل ۷: Hoisting

Hoisting رفتاری است که در آن اعلان‌ها پیش از اجرای کد به بالای محدوده‌شان منتقل (از نظر مفهومی) می‌شوند. برای تابع، این رفتار بسته به نوع تعریف متفاوت است.

Function Declaration: کاملاً Hoist می‌شود

```
console.log(sayHi("Ali")); // Hi, Ali

function sayHi(name) {
  return "Hi, " + name;
}
```

Function Expression: Hoist نمی‌شود

```
// console.log(sayBye("Ali"));
// With const/let: ReferenceError (variable is in the temporal dead zone)
// With var: TypeError (sayBye is undefined, so it is not a function)

const sayBye = function (name) {
  return "Bye, " + name;
};
console.log(sayBye("Ali")); // Bye, Ali
```

توصیه: حتی اگر قابلیت Hoisting وجود دارد، توابع را قبل از استفاده تعریف کنید تا خواندن کد ساده‌تر باشد و به رفتارهای ضمنی وابسته نشوید.

فصل ۸: دامنه (Scope) و Closure

انواع دامنه

- **Global:** متغیرهایی که بیرون از هر تابع و بلوک تعریف می‌شوند.
- **Function Scope:** متغیرهای داخل تابع فقط داخل همان تابع قابل دسترسی‌اند (`var`، `let`، `const`).
- **Block Scope:** متغیرهای `let` و `const` فقط داخل بلوک `{ }` محدود می‌شوند.

```
const globalValue = "global";

function outer() {
  const outerValue = "outer";

  function inner() {
    const innerValue = "inner";
    console.log(globalValue, outerValue, innerValue);
  }
  inner();
  // console.log(innerValue); // ReferenceError: not accessible here
}
outer(); // global outer inner
```

جاوا اسکریپت از دامنه لغوی (Lexical Scope) استفاده می‌کند: دسترسی تابع به متغیرها بر اساس محل «نوشته شدن» آن تعیین می‌شود، نه محل فراخوانی.

Closure

Closure یعنی تابع، حتی پس از پایان اجرای تابع بیرونی، به متغیرهای محیط تعریف خود دسترسی دارد. هر بار که تابع بیرونی اجرا شود، یک محیط مستقل ساخته می‌شود.

```
function createCounter() {
  let count = 0;            // private state
  return function () {
    count += 1;
    return count;
  };
}

const counterA = createCounter();
const counterB = createCounter();
console.log(counterA()); // 1
console.log(counterA()); // 2
console.log(counterB()); // 1 (independent state)
```

کاربرد: داده خصوصی

```
function createAccount(initialBalance) {
  let balance = initialBalance;
  return {
    deposit(amount) {
      if (amount <= 0) throw new Error("Amount must be positive");
      balance += amount;
      return balance;
    },
    getBalance() {
      return balance;
    },
  };
}

const account = createAccount(100);
account.deposit(50);
console.log(account.getBalance()); // 150
console.log(account.balance);      // undefined (not accessible)
```

دام کلاسیک: var در حلقه

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log("var:", i), 0);
}
// var: 3, var: 3, var: 3 (one shared variable)

for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log("let:", j), 0);
}
// let: 0, let: 1, let: 2 (a new binding per iteration)
```

نکته: به‌جای `var` از `let` و `const` استفاده کنید. `const` را پیش‌فرض قرار دهید و فقط در صورت نیاز به تغییر مقدار از `let` استفاده کنید.

فصل ۹: توابع مرتبه بالا و Callback

تابع مرتبه بالا (Higher-Order Function) تابعی است که یکی از دو کار زیر را انجام دهد: تابع دیگری را به‌عنوان ورودی بگیرد، یا تابعی را به‌عنوان خروجی برگرداند. تابعی که به‌عنوان آرگومان ارسال می‌شود، Callback نام دارد.

دریافت تابع به‌عنوان ورودی

```
function repeat(times, action) {
  for (let i = 0; i < times; i++) {
    action(i);
  }
}
repeat(3, index => console.log("Step " + index));
// Step 0, Step 1, Step 2
```

بازگرداندن تابع

```
function multiplier(factor) {
  return number => number * factor;
}
const double = multiplier(2);
const triple = multiplier(3);
console.log(double(7)); // 14
console.log(triple(7)); // 21
```

Callback همزمان و غیرهمزمان

```
// Synchronous callback: runs immediately
[1, 2, 3].forEach(n => console.log(n));

// Asynchronous callback: runs later
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");
// Output order: A, C, B
```

در حالت غیرهمزمان، callback پس از اتمام کد جاری و از طریق صف رویدادها (Event Loop) اجرا می‌شود؛ به همین دلیل ترتیب خروجی A، C، B است.

فصل ۱۰: متدهای تابعی آرایه‌ها

آرایه‌ها در جاوا اسکریپت مجموعه‌ای از متدهای مرتبه بالا دارند که پردازش داده را بدون حلقه‌های دستی ممکن می‌سازند.

متد	کاربرد	خروجی
`forEach`	اجرای یک عملیات روی هر عنصر	`undefined`
`map`	تبدیل هر عنصر به مقدار جدید	آرایه جدید هم‌اندازه
`filter`	انتخاب عناصری که شرط را برآورده می‌کنند	آرایه جدید
`reduce`	تجمیع همه عناصر در یک مقدار	یک مقدار دلخواه
`find`	اولین عنصر مطابق شرط	عنصر یا `undefined`
`some`	آیا حداقل یک عنصر مطابق است؟	`true` یا `false`
`every`	آیا همه عناصر مطابق‌اند؟	`true` یا `false`
`sort`	مرتب‌سازی (آرایه اصلی را تغییر می‌دهد)	همان آرایه

```
const numbers = [1, 2, 3, 4, 5, 6];

const doubled = numbers.map(n => n * 2);
console.log(doubled);        // [ 2, 4, 6, 8, 10, 12 ]

const evens = numbers.filter(n => n % 2 === 0);
console.log(evens);          // [ 2, 4, 6 ]

const total = numbers.reduce((acc, n) => acc + n, 0);
console.log(total);          // 21

console.log(numbers.find(n => n > 3));        // 4
console.log(numbers.some(n => n < 0));        // false
console.log(numbers.every(n => n > 0));       // true
```

زنجیره‌سازی (Chaining)

```
const result = numbers
  .filter(n => n % 2 === 0)   // [2, 4, 6]
  .map(n => n * n)            // [4, 16, 36]
  .reduce((a, b) => a + b, 0);
console.log(result); // 56
```

reduce در عمل: گروه‌بندی

```
const words = ["apple", "avocado", "banana", "blueberry", "cherry"];

const grouped = words.reduce((acc, word) => {
  const key = word[0];
  if (!acc[key]) {
    acc[key] = [];
  }
  acc[key].push(word);
  return acc;
}, {});

console.log(grouped);
// { a: [ 'apple', 'avocado' ], b: [ 'banana', 'blueberry' ], c: [ 'cherry' ] }
```

مرتب‌سازی با sort

هشدار: مرتب‌سازی پیش‌فرض، عناصر را به رشته تبدیل و به‌صورت الفبایی مقایسه می‌کند. برای اعداد حتماً تابع مقایسه بدهید. همچنین `sort` آرایه اصلی را تغییر می‌دهد.

```
console.log([10, 9, 1].sort());                 // [ 1, 10, 9 ]
console.log([10, 9, 1].sort((a, b) => a - b));  // [ 1, 9, 10 ]
console.log([10, 9, 1].sort((a, b) => b - a));  // [ 10, 9, 1 ]
```

فصل ۱۱: تابع بلافاصله اجراشونده (IIFE)

IIFE (Immediately Invoked Function Expression) تابعی است که بلافاصله پس از تعریف اجرا می‌شود. هدف اصلی آن، ایجاد یک دامنه خصوصی و جلوگیری از آلوده شدن دامنه سراسری است.

```
(function () {
  const secret = "hidden";
  console.log("IIFE runs immediately");
})();

// Arrow function version
(() => {
  console.log("Arrow IIFE");
})();

// Returning a value
const config = (function () {
  const env = "production";
  return { env, debug: false };
})();
console.log(config); // { env: 'production', debug: false }

// Async IIFE
(async () => {
  const value = await Promise.resolve(42);
  console.log(value); // 42
})();
```

نکته: با وجود ماژول‌های ES و `let/const`، استفاده از IIFE کمتر شده است، اما هنوز در اسکریپت‌های قدیمی، الگوی Module و اجرای یک‌باره کد async کاربرد دارد.

فصل ۱۲: توابع بازگشتی (Recursion)

تابع بازگشتی تابعی است که خودش را فراخوانی می‌کند. هر تابع بازگشتی دو بخش ضروری دارد: **حالت پایه (Base Case)** که بازگشت را متوقف می‌کند، و **حالت بازگشتی (Recursive Case)** که مسئله را کوچک‌تر می‌کند.

```
function factorial(n) {
  if (n < 0) throw new Error("n must be non-negative");
  if (n <= 1) return 1;          // base case
  return n * factorial(n - 1);   // recursive case
}
console.log(factorial(5)); // 120
```

ردیابی فراخوانی‌ها برای `factorial(3)`: ابتدا `3 * factorial(2)`، سپس `3 * (2 * factorial(1))`، و در نهایت `3 * 2 * 1` که برابر ۶ است.

مثال: تخت کردن آرایه تو در تو

```
function flatten(items) {
  const result = [];
  for (const item of items) {
    if (Array.isArray(item)) {
      result.push(...flatten(item));
    } else {
      result.push(item);
    }
  }
  return result;
}
console.log(flatten([1, [2, [3, [4]], 5]])); // [ 1, 2, 3, 4, 5 ]
// Built-in alternative: [1, [2, [3]]].flat(Infinity)
```

مثال: شمارش معکوس

```
function countdown(n) {
  if (n < 0) return;
  console.log(n);
  countdown(n - 1);
}
countdown(3); // 3, 2, 1, 0
```

هشدار: بازگشت بدون حالت پایه صحیح یا با عمق بسیار زیاد، خطای RangeError: Maximum call stack size exceeded ایجاد می‌کند. در مسائلی که عمق زیادی دارند، نسخه حلقه‌ای یا تکنیک Memoization را در نظر بگیرید.

تبدیل بازگشت به حلقه

```
function factorialLoop(n) {
  let result = 1;
  for (let i = 2; i <= n; i++) {
    result *= i;
  }
  return result;
}
console.log(factorialLoop(5)); // 120
```

فصل ۱۳: کلمه this و متدهای call، apply و bind

مقدار `this` در جاوا اسکریپت به نحوه «فراخوانی» تابع بستگی دارد (به‌جز توابع پیکانی). قواعد به ترتیب اولویت:

- **فراخوانی با new:** `this` شیء تازه ساخته‌شده است.
- **فراخوانی صریح با call، apply یا bind:** `this` همان مقداری است که داده شده است.
- **فراخوانی به‌عنوان متد (`obj.method()`):** `this` برابر `obj` است.
- **فراخوانی ساده (`fn()`):** در حالت strict برابر `undefined` و در غیر آن شیء سراسری است.
- **تابع پیکانی:** `this` از محیط بیرونی گرفته می‌شود.

از دست رفتن this

```
const user = {
  name: "Sara",
  greet() {
    return "Hi, " + this.name;
  },
};
console.log(user.greet()); // Hi, Sara

const detached = user.greet;
// detached() loses "this": it throws a TypeError in strict mode
// or reads the wrong object in sloppy mode
```

call، apply و bind

```
function introduce(greeting, punctuation) {
  return greeting + ", I am " + this.name + punctuation;
}
const person = { name: "Ali" };

// call: arguments listed one by one
console.log(introduce.call(person, "Hello", "!"));   // Hello, I am Ali!

// apply: arguments passed as an array
console.log(introduce.apply(person, ["Hi", "."]));   // Hi, I am Ali.

// bind: returns a new function with "this" fixed
const boundIntro = introduce.bind(person, "Hey");
console.log(boundIntro("?"));                         // Hey, I am Ali?
```

راه‌حل برای از دست رفتن this

```
const detachedFixed = user.greet.bind(user);
console.log(detachedFixed()); // Hi, Sara

// Or wrap in an arrow function
const wrapped = () => user.greet();
console.log(wrapped());       // Hi, Sara
```

فصل ۱۴: متدهای شیء و توابع سازنده

متد شیء

تابعی که به‌عنوان ویژگی یک شیء تعریف شود متد (Method) نام دارد. در ES2015 برای آن نحو کوتاه وجود دارد.

```
const calculator = {
  total: 0,
  add(n) {
    this.total += n;
    return this;      // enables chaining
  },
  subtract(n) {
    this.total -= n;
    return this;
  },
};
console.log(calculator.add(10).subtract(3).total); // 7
```

تابع سازنده (Constructor Function)

تابعی که با `new` فراخوانی می‌شود و یک شیء جدید می‌سازد. به قرارداد، نام آن با حرف بزرگ شروع می‌شود.

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}
Person.prototype.describe = function () {
  return this.name + " is " + this.age + " years old";
};

const p1 = new Person("Sara", 30);
console.log(p1.describe()); // Sara is 30 years old
```

معادل مدرن: class

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  speak() {
    return this.name + " makes a sound";
  }
}
class Dog extends Animal {
  speak() {
    return this.name + " barks";
  }
}
console.log(new Dog("Rex").speak()); // Rex barks
```

تابع کارخانه‌ای (Factory Function)

روشی بدون `new` و `this`؛ شیء را می‌سازد و برمی‌گرداند و از Closure برای داده خصوصی استفاده می‌کند.

```
function createPerson(name, age) {
  return {
    name,
    age,
    describe() {
      return name + " is " + age + " years old";
    },
  };
}
console.log(createPerson("Ali", 25).describe()); // Ali is 25 years old
```

فصل ۱۵: توابع غیرهمزمان: Promise و async/await

عملیات‌هایی مانند درخواست شبکه، خواندن فایل یا تایمر نتیجه خود را بعداً برمی‌گردانند. جاوا اسکریپت برای مدیریت آن‌ها سه الگو دارد: Callback، Promise و async/await.

Callback

```
function loadData(callback) {
  setTimeout(() => {
    callback(null, { id: 1 });
  }, 500);
}
loadData((error, data) => {
  if (error) return console.error(error);
  console.log(data); // { id: 1 }
});
```

تو در تو شدن callbackها به «Callback Hell» منجر می‌شود؛ Promise برای حل همین مشکل معرفی شد.

Promise

```
function delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

delay(500)
  .then(() => {
    console.log("500 ms passed");
    return "next value";
  })
  .then(value => console.log(value))
  .catch(error => console.error(error))
  .finally(() => console.log("done"));
```

async و await

```
async function run() {
  console.log("start");
  await delay(500);
  console.log("end");
  return "finished";
}
run().then(result => console.log(result));
// start, (500 ms later) end, finished
```

- تابع `async` همیشه یک Promise برمی‌گرداند.
- `await` فقط داخل تابع `async` (یا در سطح بالای ماژول‌های ES) مجاز است.
- `await` اجرای همان تابع را متوقف می‌کند، نه کل برنامه را.

مدیریت خطا با try/catch

```
async function getUser(id) {
  try {
    const response = await fetch("https://api.example.com/users/" + id);
    if (!response.ok) {
      throw new Error("HTTP error: " + response.status);
    }
    return await response.json();
  } catch (error) {
    console.error("Failed to load user:", error.message);
    return null;
  }
}
```

اجرای موازی با Promise.all

```
async function loadAll() {
  const [a, b] = await Promise.all([
    delay(100).then(() => "A"),
    delay(200).then(() => "B"),
  ]);
  console.log(a, b); // A B (total time about 200 ms, not 300 ms)
}
loadAll();
```

هشدار: اگر عملیات‌ها به هم وابسته نیستند، آن‌ها را پشت سر هم `await` نکنید؛ با `Promise.all` همزمان اجرا کنید تا زمان کل کاهش یابد. توجه کنید که `Promise.all` با اولین خطا رد (Reject) می‌شود.

فصل ۱۶: توابع Generator

Generator تابعی است که می‌تواند اجرای خود را متوقف کند و بعداً از همان نقطه ادامه دهد. با `functio` `n*` تعریف می‌شود و با `yield` مقدار تولید می‌کند. نتیجه فراخوانی، یک iterator است.

```
function* idGenerator() {
  let id = 1;
  while (true) {
    yield id++;
  }
}
const gen = idGenerator();
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
console.log(gen.next());       // { value: 3, done: false }
```

```
function* range(start, end) {
  for (let i = start; i <= end; i++) {
    yield i;
  }
}
console.log([...range(1, 5)]);   // [ 1, 2, 3, 4, 5 ]
for (const n of range(1, 3)) {
  console.log(n);                // 1, 2, 3
}
```

نکته: Generatorها برای تولید دنباله‌های بزرگ یا بی‌پایان به‌صورت تنبل (Lazy)، پیمایش سفارشی و پیاده‌سازی بسیاری از کتابخانه‌های جریان داده استفاده می‌شوند.

فصل ۱۷: الگوهای پیشرفته در توابع

Currying

تبدیل تابعی با چند پارامتر به زنجیره‌ای از توابع تک‌پارامتری.

```
const curriedAdd = a => b => c => a + b + c;
console.log(curriedAdd(1)(2)(3)); // 6

const addTen = curriedAdd(10);
console.log(addTen(5)(5));        // 20
```

ترکیب توابع (Composition و Pipe)

```
const pipe = (...fns) => input => fns.reduce((acc, fn) => fn(acc), input);

const trim = s => s.trim();
const toUpper = s => s.toUpperCase();
const exclaim = s => s + "!";

const shout = pipe(trim, toUpper, exclaim);
console.log(shout("  hello  ")); // HELLO!
```

Memoization

ذخیره نتیجه فراخوانی‌های قبلی برای جلوگیری از محاسبه مجدد. این روش فقط برای توابع خالص مناسب است.

```
function memoize(fn) {
  const cache = new Map();
  return function (arg) {
    if (cache.has(arg)) {
      return cache.get(arg);
    }
    const result = fn(arg);
    cache.set(arg, result);
    return result;
  };
}

const fib = memoize(n => (n < 2 ? n : fib(n - 1) + fib(n - 2)));
console.log(fib(50)); // 12586269025
```

Debounce

اجرای تابع فقط پس از گذشت مدت مشخص از آخرین فراخوانی؛ مناسب برای جستجوی زنده و رویداد تغییر اندازه پنجره.

```
function debounce(fn, wait) {
  let timerId;
  return function (...args) {
    clearTimeout(timerId);
    timerId = setTimeout(() => fn.apply(this, args), wait);
  };
}

const onSearch = debounce(query => console.log("search:", query), 300);
onSearch("j");
onSearch("ja");
onSearch("jav"); // only this call runs, 300 ms later
```

Once

```
function once(fn) {
  let called = false;
  let result;
  return function (...args) {
    if (!called) {
      called = true;
      result = fn.apply(this, args);
    }
    return result;
  };
}
const init = once(() => {
  console.log("initialized");
  return 1;
});
init(); // initialized
init(); // (nothing printed)
```

فصل ۱۸: مدیریت خطا در توابع

تابع باید در برابر ورودی نامعتبر و شرایط غیرمنتظره رفتار مشخصی داشته باشد. ابزار اصلی، دستور `thro` `w` و ساختار `try/catch/finally` است.

```
function parseAge(input) {
  const age = Number(input);
  if (!Number.isInteger(age) || age < 0) {
    throw new RangeError("Age must be a non-negative integer");
  }
  return age;
}

try {
  parseAge("abc");
} catch (error) {
  console.error(error.name + ": " + error.message);
  // RangeError: Age must be a non-negative integer
} finally {
  console.log("always runs");
}
```

خطای سفارشی

```
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = "ValidationError";
    this.field = field;
  }
}

function validateUser(user) {
  if (!user.email) {
    throw new ValidationError("email", "Email is required");
  }
  return true;
}

try {
  validateUser({});
} catch (error) {
  if (error instanceof ValidationError) {
    console.log(error.field + ": " + error.message); // email: Email is required
  } else {
    throw error; // unknown errors are re-thrown
  }
}
```

چه زمانی throw کنیم و چه زمانی مقدار بازگردانیم؟

- برای خطاهای برنامه‌نویسی و شرایط واقعاً استثنایی (ورودی نامعتبر، نقض پیش‌شرط) از `throw` استفاده کنید.
- برای حالت‌های عادی و قابل انتظار (مثلاً «نتیجه‌ای پیدا نشد») بازگرداندن `null` یا مقدار مشخص مناسب‌تر است.
- هرگز `catch` خالی ننویسید؛ خطای بلعیده‌شده، عیب‌یابی را بسیار دشوار می‌کند.

فصل ۱۹: توابع خالص و اثرات جانبی

تابع خالص (Pure Function) دو ویژگی دارد: به ازای ورودی یکسان همیشه خروجی یکسان می‌دهد، و هیچ اثر جانبی (Side Effect) ندارد؛ یعنی چیزی بیرون از خودش را تغییر نمی‌دهد و به حالت بیرونی وابسته نیست.

```
// Impure: depends on and changes external state
let total = 0;
function addToTotal(n) {
  total += n;
  return total;
}

// Pure: output depends only on input
function add(a, b) {
  return a + b;
}

// Impure: mutates its argument
function addItemMutating(list, item) {
  list.push(item);
  return list;
}

// Pure: returns a new array
function addItem(list, item) {
  return [...list, item];
}

const original = [1, 2];
const updated = addItem(original, 3);
console.log(original); // [ 1, 2 ]
console.log(updated);  // [ 1, 2, 3 ]
```

نمونه‌هایی از اثرات جانبی

- تغییر متغیر سراسری یا آرگومان ورودی
- نوشتن در کنسول، فایل یا پایگاه داده
- درخواست شبکه
- استفاده از `Math.random()` یا `Date.now()` (خروجی وابسته به شرایط بیرونی)

مزایا

- پیش‌بینی‌پذیری و سهولت عیب‌یابی
- آزمون‌پذیری بسیار ساده (بدون نیاز به آماده‌سازی محیط)
- امکان استفاده از Memoization و اجرای موازی بی‌خطر

نکته: برنامه واقعی بدون اثر جانبی کاری انجام نمی‌دهد. هدف حذف کامل آن‌ها نیست؛ هدف جدا کردن منطق خالص از بخش‌های دارای اثر جانبی و محدود کردن آن‌ها به لبه‌های برنامه است.

فصل ۲۰: اصول و بهترین روش‌ها

اصول طراحی تابع

- **مسئولیت واحد:** هر تابع یک کار مشخص انجام دهد.
- **کوتاه بودن:** تابعی که از یک صفحه بلندتر شود معمولاً باید شکسته شود.
- **نام توصیفی:** نام باید هدف را بیان کند، نه پیاده‌سازی را.
- **تعداد کم پارامتر:** بیش از سه پارامتر را به یک شیء تبدیل کنید.
- **بازگشت زودهنگام:** از تو در تو شدن بیش از حد شرط‌ها پرهیز کنید.
- **اجتناب از اثر جانبی:** هر جا ممکن است تابع خالص بنویسید.
- **عدم تغییر آرگومان‌ها:** به‌جای تغییر ورودی، مقدار جدید بسازید.
- **اعداد و رشته‌های ثابت را نام‌گذاری کنید:** از «اعداد جادویی» (Magic Numbers) استفاده نکنید.
- **از `===` استفاده کنید:** مقایسه با تبدیل نوع ضمنی (==) خطاخیز است.

مستندسازی با JSDoc

```
/**
 * Calculates the final price including tax.
 * @param {number} price - Base price.
 * @param {number} [taxRate=0.1] - Tax rate between 0 and 1.
 * @returns {number} Final price.
 */
function priceWithTax(price, taxRate = 0.1) {
  return price * (1 + taxRate);
}
console.log(priceWithTax(200)); // 220.00000000000003 (floating point)
```

نکته: نتیجه نمایش‌داده‌شده به دلیل ماهیت اعداد اعشاری در استاندارد IEEE ۷۵۴ است. در محاسبات مالی از اعداد صحیح (مثلاً کوچک‌ترین واحد پول) یا کتابخانه‌های مخصوص استفاده کنید.

خطاهای رایج

- فراموش کردن `return` و دریافت `undefined`
- نوشتن `fn` به‌جای `fn()` یا برعکس؛ به‌خصوص هنگام ارسال callback: `setTimeout(run(), 100)` تابع را همان لحظه اجرا می‌کند، نه بعد از ۱۰۰ میلی‌ثانیه
- استفاده از تابع پیکانی به‌عنوان متد شیء یا سازنده
- استفاده از `var` در حلقه‌هایی که Closure می‌سازند
- تغییر ناخواسته آرگومان‌های شیء و آرایه
- فراموش کردن `await` و کار با Promise به‌جای مقدار نهایی
- استفاده مستقیم از توابعی که چند آرگومان می‌گیرند به‌عنوان callback:

```
console.log(["1", "2", "3"].map(parseInt)); // [ 1, NaN, NaN ]
console.log(["1", "2", "3"].map(Number));   // [ 1, 2, 3 ]
// map passes (value, index, array); parseInt treats index as the radix
```

فصل ۲۱: تمرین‌ها

تمرین‌ها را پیش از مراجعه به فصل بعد، خودتان حل کنید و سپس با پاسخ‌ها مقایسه نمایید. سطح هر تمرین در پرانتز آمده است.

- **تمرین ۱:** (مقدماتی) تابع `isEven(n)` را بنویسید که برای عدد زوج `true` و در غیر این صورت `fal` `se` برگرداند.
- **تمرین ۲:** (مقدماتی) تابع `max(...numbers)` را با پارامتر rest بنویسید که بزرگ‌ترین عدد را برگرداند. اگر هیچ عددی داده نشد، `null` برگرداند.
- **تمرین ۳:** (مقدماتی) تابع `countVowels(text)` را بنویسید که تعداد حروف صدادار انگلیسی (a, e, i, o, u) در یک رشته را برگرداند.
- **تمرین ۴:** (متوسط) با استفاده از Closure، تابع `createCounter(start)` را بنویسید که شیئی با متدهای `reset`، `decrement`، `increment` و `get` برگرداند.
- **تمرین ۵:** (متوسط) با استفاده از `reduce`، تابع `average(numbers)` را بنویسید. برای آرایه خالی مقدار صفر برگردانده شود.
- **تمرین ۶:** (متوسط) تابع `capitalizeWords(text)` را بنویسید که حرف اول هر کلمه را بزرگ کند. مثال: `hello big world` به `Hello Big World` تبدیل شود.
- **تمرین ۷:** (متوسط) تابع بازگشتی `sumDigits(n)` را بنویسید که مجموع ارقام یک عدد صحیح را برگرداند.
- **تمرین ۸:** (متوسط) تابع `isPalindrome(text)` را بنویسید که بدون توجه به حروف بزرگ و کوچک و نمادها، متقارن بودن رشته را بررسی کند.
- **تمرین ۹:** (پیشرفته) تابع `groupBy(items, keyFn)` را با `reduce` بنویسید که عناصر را بر اساس کلید برگشتی از `keyFn` گروه‌بندی کند.
- **تمرین ۱۰:** (پیشرفته) تابع `retry(task, times)` را بنویسید که یک تابع غیرهمزمان را حداکثر `ti` `mes` بار اجرا کند و در صورت موفقیت نتیجه، و در صورت شکست همه تلاش‌ها آخرین خطا را برگرداند (throw کند).

فصل ۲۲: پاسخ تمرین‌ها

پاسخ تمرین ۱

```
const isEven = n => n % 2 === 0;
console.log(isEven(4)); // true
console.log(isEven(7)); // false
```

پاسخ تمرین ۲

```
function max(...numbers) {
  if (numbers.length === 0) return null;
  return numbers.reduce((best, n) => (n > best ? n : best));
}
console.log(max(3, 9, 4)); // 9
console.log(max());        // null
```

پاسخ تمرین ۳

```
function countVowels(text) {
  const matches = text.toLowerCase().match(/[aeiou]/g);
  return matches ? matches.length : 0;
}
console.log(countVowels("JavaScript")); // 3
```

پاسخ تمرین ۴

```
function createCounter(start = 0) {
  let value = start;
  return {
    increment: () => ++value,
    decrement: () => --value,
    reset: () => (value = start),
    get: () => value,
  };
}
const c = createCounter(5);
c.increment();
c.increment();
console.log(c.get());   // 7
console.log(c.reset()); // 5
```

پاسخ تمرین ۵

```
const average = numbers =>
  numbers.length === 0
    ? 0
    : numbers.reduce((sum, n) => sum + n, 0) / numbers.length;

console.log(average([2, 4, 6])); // 4
console.log(average([]));        // 0
```

پاسخ تمرین ۶

```
function capitalizeWords(text) {
  return text
    .split(" ")
    .map(word => (word ? word[0].toUpperCase() + word.slice(1) : word))
    .join(" ");
}
console.log(capitalizeWords("hello big world")); // Hello Big World
```

پاسخ تمرین ۷

```
function sumDigits(n) {
  n = Math.abs(n);
  if (n < 10) return n;                          // base case
  return (n % 10) + sumDigits(Math.floor(n / 10));
}
console.log(sumDigits(1234)); // 10
```

پاسخ تمرین ۸

```
function isPalindrome(text) {
  const cleaned = text.toLowerCase().replace(/[^a-z0-9]/g, "");
  return cleaned === [...cleaned].reverse().join("");
}
console.log(isPalindrome("A man, a plan, a canal: Panama")); // true
console.log(isPalindrome("hello"));                          // false
```

پاسخ تمرین ۹

```
function groupBy(items, keyFn) {
  return items.reduce((acc, item) => {
    const key = keyFn(item);
    if (!acc[key]) {
      acc[key] = [];
    }
    acc[key].push(item);
    return acc;
  }, {});
}
console.log(groupBy([1, 2, 3, 4, 5], n => (n % 2 === 0 ? "even" : "odd")));
// { odd: [ 1, 3, 5 ], even: [ 2, 4 ] }
```

پاسخ تمرین ۱۰

```
async function retry(task, times) {
  let lastError;
  for (let attempt = 1; attempt <= times; attempt++) {
    try {
      return await task();
    } catch (error) {
      lastError = error;
    }
  }
  throw lastError;
}

// Usage example: fails twice, then succeeds
let calls = 0;
const flaky = async () => {
  calls += 1;
  if (calls < 3) throw new Error("temporary failure");
  return "ok after " + calls + " calls";
};
retry(flaky, 5).then(console.log); // ok after 3 calls
```

پیوست: جدول مقایسه انواع تابع

نوع تابع	Hoisting	this	arguments	new	نکته
Function Declaration	کامل	مخصوص خود	دارد	بله	رایج‌ترین روش تعریف
Function Expression	خیر	مخصوص خود	دارد	بله	مقدار قابل انتقال
Arrow Function	خیر	از محیط بیرونی	ندارد	خیر	مناسب callback کوتاه
Method (نحو کوتاه)	خیر	شیء صاحب متد	دارد	خیر	داخل شیء و کلاس
Generator (`function*`)	مانند Declaration	مخصوص خود	دارد	خیر	تولید دنباله مقادیر
`async function`	مطابق نوع تعریف	مخصوص خود	دارد	خیر	همیشه Promise برمی‌گرداند

خلاصه نکات کلیدی

- تابع در جاوا اسکریپت یک مقدار است و می‌توان آن را ذخیره، ارسال و بازگرداند.
- برای مقدار پیش‌فرض از `=`، برای تعداد نامحدود آرگومان از `...rest` و برای پارامتر نام‌دار از Destructuring استفاده کنید.
- `const` و `let` را بر `var` ترجیح دهید و به تفاوت Hoisting در Declaration و Expression توجه کنید.
- Closure به تابع اجازه می‌دهد به محیط تعریف خود دسترسی داشته باشد و پایه الگوهایی مانند داده خصوصی و Memoization است.
- تابع پیکانی `this` ندارد؛ برای callback مناسب است و برای متد شیء و سازنده مناسب نیست.
- `filter`، `map` و `reduce` را برای پردازش داده به حلقه‌های دستی ترجیح دهید.
- برای عملیات غیرهمزمان از `async/await` و برای اجرای موازی از `Promise.all` استفاده کنید.
- توابع کوچک، خالص و با نام گویا بنویسید و خطاها را به‌صورت صریح مدیریت کنید.

منابع تکمیلی

- MDN Web Docs: بخش Functions و Guide (developer.mozilla.org)
- ECMAScript Language Specification (tc39.es/ecma262)
- javascript.info: بخش‌های Functions و Closures